

LC3 Navigator Admin Guide

LC3 Navigator - Admin Guide: Config Format และการอัปเดตระบบ

Task: Task1: US6 (System Design)
Project: CS232-Map_Navigation / LC3 Navigator
Purpose: คู่มือนี้ใช้สำหรับ Admin ที่ต้องการแก้ไขข้อมูลห้อง เส้นทาง รายวิชา event และรูปแบบผังชั้น โดยไม่ต้องแก้ logic หลักของระบบ

1. ภาพรวมตาม Architecture ล่าสุด

ระบบ LC3 Navigator ใช้ AWS Serverless Architecture ตามที่ออกแบบไว้ โดยแบ่งข้อมูลออกเป็น 2 bucket หลัก และใช้ Lambda 3 ตัวในการทำงาน

ส่วนของระบบ	Resource จริง	หน้าที่
Static Website Bucket	lc3-navigator-static-staging	เก็บไฟล์หน้าเว็บ เช่น HTML, CSS, JavaScript
Config Bucket	lc3-navigator-config-staging	เก็บไฟล์ config เช่น graph.json , search_metadata.json และ floor plan
Search Lambda	lc3-search-staging	ค้นหาห้อง วิชา event และสถานที่จาก DynamoDB
Pathfinding Lambda	lc3-pathfinding-staging	คำนวณเส้นทางจาก graph config
Direction Lambda	lc3-direction-staging	แปลง path เป็นคำแนะนำการเดินทาง
DynamoDB	LC3_CourseMapping_staging	เก็บ search index สำหรับค้นหา
GitHub Actions	Deploy to Staging	Deploy config, frontend, lambda และ seed DynamoDB อัตโนมัติ

สรุป flow การทำงานหลัง Admin แก้ config คือ

```
Admin แก้ไฟล์ config ใน repo
-> push เข้า branch dev
-> GitHub Actions ตรวจ JSON
-> upload config ไป S3 config bucket
-> seed ข้อมูลลง DynamoDB
-> deploy frontend ไป static bucket
-> deploy Lambda ทั้ง 3 ตัว
-> update CONFIG_VERSION เพื่อให้ Lambda โหลด config ล่าสุด
```

2. ไฟล์ Config ที่ Admin ต้องรู้จัก

ไฟล์ / Folder	หน้าที่	ถูก deploy ไปที่
Database/graph.json	เก็บ node, edge, ห้อง, ชั้น, พิกัด และเส้นทาง	s3://lc3-navigator-config-staging/graph/graph.json
Database/search_metadata.json	เก็บ mapping ของ course / event ไปยัง node หรือห้อง	s3://lc3-navigator-config-staging/config/search_metadata.json
Frontend/resources/	เก็บรูป floor plan เช่น PNG, JPG, SVG	s3://lc3-navigator-config-staging/floor-plans/
Database/config_format/	เก็บ schema อธิบาย format ของ config	ใช้เป็นเอกสารอ้างอิงใน repo
Database/config_examples/	เก็บตัวอย่าง config สำหรับ Admin	ใช้เป็นตัวอย่างก่อนแก้ไฟล์จริง

หมายเหตุ: ไฟล์ใน Database/config_format/ และ Database/config_examples/ เป็นไฟล์ช่วยอธิบาย ไม่ใช่ไฟล์หลักที่ Lambda ใช้โดยตรง

3. Config Format: Database/graph.json

graph.json คือไฟล์แผนที่ของอาคารในรูปแบบ graph ใช้สำหรับ pathfinding และ direction

โครงสร้างหลักของไฟล์คือ

```
{
  "building": "LC3",
  "nodes": [],
  "edges": []
}
```

3.1 Field: building

Field	Type	Required	ตัวอย่าง	คำอธิบาย
building	string	yes	LC3	ชื่ออาคาร

3.2 Field: nodes

nodes คือรายการจุดบนแผนที่ เช่น ห้องเรียน ทางเข้า ทางเดิน บันได และจุดสำคัญ

ตัวอย่าง node จริงในระบบ

```
{
  "id": "LC3_111",
  "name": "111",
  "floor": 1,
  "x": 83,
  "y": 169,
  "type": "room",
  "label": "ห้องบรรยายเรียนรวม"
}
```

Field	Type	Required	ตัวอย่าง	คำอธิบาย
id	string	yes	LC3_111	รหัส node ต้องไม่ซ้ำกัน

Field	Type	Required	ตัวอย่าง	คำอธิบาย
name	string	yes	111	ชื่อที่ใช้แสดงหรือใช้ค้นหา
floor	number	yes	1	ชั้นของ node
x	number	yes	83	พิกัดแกน X บนรูป floor plan
y	number	yes	169	พิกัดแกน Y บนรูป floor plan
type	string	yes	room	ประเภท node เช่น room , entrance , junction , stairs , facility
label	string	optional	ห้องบรรยายเรียนรวม	คำอธิบายเพิ่มเติม

ประเภท node ที่ใช้ในระบบปัจจุบันมี เช่น

room, entrance, junction, stairs, facility

3.3 Field: edges

edges คือเส้นเชื่อมระหว่าง node ใช้บอกว่าจุดใดเดินเชื่อมถึงจุดใดได้

ตัวอย่าง edge จริงในระบบ

```
{
  "from": "LC3_F2_hallway-1",
  "to": "LC3_F2_entry-32",
  "type": "walk",
  "distance": 0
}
```

Field	Type	Required	ตัวอย่าง	คำอธิบาย
from	string	yes	LC3_F2_hallway-1	node เริ่มต้น
to	string	yes	LC3_F2_entry-32	node ปลายทาง
type	string	yes	walk	ประเภทเส้นทาง เช่น walk , stairs , up , down
distance	number	yes	0	ระยะทางหรือค่าน้ำหนักของเส้นทาง

ข้อสำคัญ: ค่า from และ to ต้องเป็น node ที่มีอยู่จริงใน nodes

3.4 ตัวอย่างการเพิ่มห้องใหม่ใน graph.json

ถ้าต้องการเพิ่มห้อง 232 ชั้น 2 ให้ทำ 2 ส่วน

1. เพิ่ม node ใหม่ใน nodes

```
{
  "id": "LC3_F2_232",
  "name": "232",
  "floor": 2,
  "x": 780,
  "y": 340,
  "type": "room",
  "label": "ห้องเรียน"
}
```

1. เพิ่ม edge เชื่อมห้องกับทางเดินใกล้สุด

```
{
  "from": "LC3_F2_232",
  "to": "LC3_F2_hallway-12",
  "type": "walk",
}
```

```
"distance": 6
}
```

4. Config Format: Database/search_metadata.json

search_metadata.json ใช้สำหรับเพิ่มข้อมูลที่ต้องการให้ search เจอ เช่น รายวิชาและ event

โครงสร้างไฟล์เป็น array ของ object

```
[
  {
    "category": "course",
    "course_id": "CS111",
    "sec": "650001",
    "node_id": "122"
  },
  {
    "category": "event",
    "name": "Open House (จุดลงทะเบียน)",
    "node_id": "hallway-5"
  }
]
```

4.1 Course Mapping

ใช้เมื่อ Admin ต้องการให้ user ค้นหาด้วยรหัสวิชาและ section

```
{
  "category": "course",
  "course_id": "CS111",
  "sec": "650001",
  "node_id": "122"
}
```

Field	Type	Required	คำอธิบาย
category	string	yes	ต้องเป็น course
course_id	string	yes	รหัสวิชา เช่น CS111
sec	string	yes	section เช่น 650001
node_id	string	yes	ห้องหรือ node ปลายทาง ต้อง resolve ได้จาก graph.json

ระบบ seed จะพยายามหา node_id จาก id ก่อน และถ้าไม่เจอจะลองหาโดยใช้ name ของ node ดังนั้นในระบบนี้ node_id อาจเป็นได้ทั้ง LC3_122 หรือ 122 ถ้าใน graph.json มี node name เป็น 122

4.2 Event Mapping

ใช้เมื่อ Admin ต้องการให้ user ค้นหาด้วยชื่อ event

```
{
  "category": "event",
  "name": "Open House (จุดลงทะเบียน)",
  "node_id": "hallway-5"
}
```

Field	Type	Required	คำอธิบาย
category	string	yes	ต้องเป็น event
name	string	yes	ชื่อ event ที่ user จะค้นหา
node_id	string	yes	ห้องหรือ node ปลายทาง ต้อง resolve ได้จาก graph.json

5. Config Format Reference Files

ใน package นี้มีไฟล์ format และตัวอย่าง config ให้ใช้ประกอบการแก้ไขไฟล์จริง

5.1 Database/config_format/graph.schema.json

ใช้เป็น reference ของ format graph.json

ช่วยอธิบายว่า graph ต้องมี

- building
- nodes
- edges

และแต่ละ node / edge ควรมี field อะไรบ้าง

5.2 Database/config_format/search_metadata.schema.json

ใช้เป็น reference ของ format search_metadata.json

ช่วยอธิบายว่า search metadata รองรับ 2 category หลัก

- course
 - event
-

5.3 Database/config_examples/graph_node_edge.example.json

เป็นตัวอย่างการเพิ่ม node และ edge

ใช้ดูเป็นต้นแบบก่อนแก้ไข Database/graph.json

5.4 Database/config_examples/search_metadata.example.json

เป็นตัวอย่างการเพิ่ม course และ event

ใช้ดูเป็นต้นแบบก่อนแก้ไข Database/search_metadata.json

6. ขั้นตอนการอัปเดต Config สำหรับ Admin

ขั้นตอนที่ 1: เปิดโปรเจกต์

```
code .
```

ขั้นตอนที่ 2: แก้ไฟล์ที่ต้องการ

ต้องการแก้ไขอะไร	แก้ไฟล์
ห้อง เส้นทาง node edge	Database/graph.json
รายวิชา event search metadata	Database/search_metadata.json
รูปแผนผังชั้น	Frontend/resources/

ขั้นตอนที่ 3: ตรวจ JSON ก่อน push

```
python3 -m json.tool Database/graph.json > /dev/null
python3 -m json.tool Database/search_metadata.json > /dev/null
```

ถ้าไม่มี error แปลว่า JSON format ถูกต้อง

ขั้นตอนที่ 4: เช็กไฟล์ที่แก้

```
git status
```

ขั้นตอนที่ 5: Commit

กรณีแก้ config JSON

```
git add Database/graph.json Database/search_metadata.json
git commit -m "Update navigation config"
```

กรณีแก้ floor plan ด้วย

```
git add Database/graph.json Database/search_metadata.json Frontend/resources
git commit -m "Update navigation config and floor plans"
```

ขั้นตอนที่ 6: Push เข้า branch dev

```
git push origin dev
```

หลังจาก push แล้ว GitHub Actions จะ deploy อัตโนมัติ

7. สิ่งที่ GitHub Actions ทำให้อัตโนมัติ

เมื่อ push เข้า branch dev workflow Deploy to Staging จะทำงานดังนี้

1. Validate Database/graph.json
 2. Validate Database/search_metadata.json
 3. Upload Database/graph.json ไปที่ s3://lc3-navigator-config-staging/graph/graph.json
 4. Upload Database/search_metadata.json ไปที่ s3://lc3-navigator-config-staging/config/search_metadata.json
 5. Upload Frontend/resources/ ไปที่ s3://lc3-navigator-config-staging/floor-plans/
 6. Seed ข้อมูลใหม่ลง DynamoDB table LC3_CourseMapping_staging
 7. Deploy Frontend ไปที่ lc3-navigator-static-staging
 8. Deploy Lambda ทั้ง 3 ตัว
 9. Update CONFIG_VERSION ให้ Lambda โหลด config ล่าสุด
-

8. วิธีตรวจสอบหลัง Deploy

8.1 ตรวจสอบ GitHub Actions

ไปที่ GitHub Repository แล้วเปิด

Actions > Deploy to Staging

ต้องเห็น job เป็นสีเขียวทั้งหมด

Deploy Config to S3 and DynamoDB
Deploy Frontend to Static S3
Deploy Lambda Functions

8.2 ตรวจสอบ Config Bucket

```
aws s3 ls s3://lc3-navigator-config-staging/graph/  
aws s3 ls s3://lc3-navigator-config-staging/config/
```

ควรเห็นไฟล์

```
graph.json  
search_metadata.json
```

8.3 ตรวจสอบ DynamoDB

```
aws dynamodb scan \  
--table-name LC3_CourseMapping_staging \  
--select COUNT
```

ควรเห็นจำนวนข้อมูลมากกว่า 0 เช่น

```
{
  "Count": 260,
  "ScannedCount": 260,
  "ConsumedCapacity": null
}
```

8.4 ດຽວ Lambda Environment Variables

Search Lambda

```
aws lambda get-function-configuration \
  --function-name lc3-search-staging \
  --query "Environment.Variables" \
  --output json
```

ຄວາມ

```
{
  "DDB_TABLE": "LC3_CourseMapping_staging",
  "CONFIG_VERSION": "commit-id"
}
```

Pathfinding Lambda

```
aws lambda get-function-configuration \
  --function-name lc3-pathfinding-staging \
  --query "Environment.Variables" \
  --output json
```

ຄວາມ

```
{
  "GRAPH_S3_BUCKET": "lc3-navigator-config-staging",
  "GRAPH_S3_KEY": "graph/graph.json",
}
```

```
"CONFIG_VERSION": "commit-id"
}
```

Direction Lambda

```
aws lambda get-function-configuration \
--function-name lc3-direction-staging \
--query "Environment.Variables" \
--output json
```

ควรมี

```
{
  "GRAPH_S3_BUCKET": "lc3-navigator-config-staging",
  "GRAPH_S3_KEY": "graph/graph.json",
  "CONFIG_VERSION": "commit-id"
}
```

9. วิธีทดสอบจากหน้าเว็บ

Test 1: ค้นหาห้อง

1. เปิดหน้าเว็บ LC3 Navigator
2. ค้นหาห้องที่มีอยู่ใน `graph.json`
3. ระบบต้องแสดงผลลัพท์ห้องที่ค้นหา

Expected Result

ระบบค้นหาห้องเจอ และแสดงข้อมูลถูกต้อง

Test 2: ค้นหาวิชา

1. เพิ่มหรือแก้ไข course ใน `search_metadata.json`

2. Push เข้า branch `dev`
3. รอ GitHub Actions deploy สำเร็จ
4. เปิดหน้าเว็บแล้วค้นหารหัสวิชาหรือชื่อวิชา

Expected Result

ระบบค้นหาวิชาเจอ และแสดงห้องที่ map กับวิชานั้นถูกต้อง

Test 3: ค้นหา Event

1. เพิ่มหรือแก้ไข event ใน `search_metadata.json`
2. Push เข้า branch `dev`
3. รอ GitHub Actions deploy สำเร็จ
4. เปิดหน้าเว็บแล้วค้นหาชื่อ event

Expected Result

ระบบค้นหา event เจอ และแสดงห้องของ event ถูกต้อง

Test 4: Pathfinding

1. เลือกจุดเริ่มต้น
2. เลือกจุดปลายทาง
3. กดค้นหาเส้นทาง

Expected Result

ระบบแสดงเส้นทางตาม `graph.json` ล่าสุด

Test 5: Direction

1. ค้นหาเส้นทาง
2. ตรวจสอบคำแนะนำการเดินทาง

Expected Result

10. Troubleshooting

ปัญหา 1: GitHub Actions fail ที่ Validate config JSON files

สาเหตุที่เป็นไปได้

- JSON มี comma เกิน
- ลืมปิด } หรือ]
- ใช้ quote ผิด
- แก้ไฟล์แล้ว structure ไม่ถูกต้อง

วิธีแก้

```
python3 -m json.tool Database/graph.json
python3 -m json.tool Database/search_metadata.json
```

ปัญหา 2: Search ไม่เจอข้อมูลใหม่

ให้เช็คตามลำดับ

1. GitHub Actions ผ่านไหม
2. search_metadata.json ถูก upload ไป config bucket แล้วไหม
3. DynamoDB มีข้อมูลไหมไหม
4. Search Lambda ใช้ table ถูกไหม

คำสั่งเช็ค DynamoDB

```
aws dynamodb scan \
--table-name LC3_CourseMapping_staging \
--select COUNT
```

คำสั่งเช็ค Search Lambda

```
aws lambda get-function-configuration \  
--function-name lc3-search-staging \  
--query "Environment.Variables" \  
--output json
```

ปัญหา 3: Pathfinding ยังใช้ graph เก่า

ให้เช็คค่า Lambda มี CONFIG_VERSION ล่าสุดหรือไม่

```
aws lambda get-function-configuration \  
--function-name lc3-pathfinding-staging \  
--query "Environment.Variables" \  
--output json
```

ถ้า CONFIG_VERSION เปลี่ยนตาม commit ล่าสุด แปลว่า Lambda ถูก refresh แล้ว

ปัญหา 4: เปิดเว็บไซต์แล้ว frontend ไม่อัปเดต

ให้ลอง hard refresh browser

Mac: Command + Shift + R
Windows: Ctrl + F5

ถ้ายังไม่อัปเดต ให้เช็ค static bucket

```
aws s3 ls s3://lc3-navigator-static-staging/
```

11. Checklist ก่อน Push

ก่อน push ทุกครั้ง Admin ควรตรวจว่า

- ☐ แก้ไฟล์ถูกต้องที่ เช่น Database/graph.json หรือ Database/search_metadata.json
 - ☐ JSON format ถูกต้อง
 - ☐ node id ไม่ซ้ำกัน
 - ☐ edge อ้างถึง node ที่มีอยู่จริง
 - ☐ course / event อ้างถึง node ที่ resolve ได้จาก graph.json
 - ☐ commit message สื่อความหมาย
 - ☐ GitHub Actions หลัง push ต้องเป็นสีเขียว
-

12. Definition of Done

งาน US6 ถือว่าเสร็จเมื่อ

- Admin สามารถดู format ของ config ได้จากเอกสารนี้
 - Admin มีตัวอย่าง config ให้อ่าน Database/config_examples/
 - Admin สามารถแก้ graph.json หรือ search_metadata.json ได้
 - Admin สามารถ push เข้า branch dev ได้
 - GitHub Actions deploy สำเร็จ
 - Config ถูก upload ไป S3 config bucket
 - DynamoDB ถูก seed ใหม่จาก config ล่าสุด
 - Lambda ทั้ง 3 ตัวใช้ resource ถูกต้องตาม architecture
 - หน้าเว็บค้นหาและนำทางจากข้อมูลล่าสุดได้
-

13. สรุปแบบสั้นที่สุด

1. เปิด *project*

code .

2. แก้ *config*

code Database/graph.json

code Database/search_metadata.json

3. ตรวจสอบ *JSON*

python3 -m json.tool Database/graph.json > /dev/null

python3 -m json.tool Database/search_metadata.json > /dev/null

4. *commit*

git add Database/graph.json Database/search_metadata.json Frontend/resources

git commit -m "Update LC3 navigation config"

5. *deploy*

git push origin dev

หลัง push ให้ตรวจที่ GitHub Actions ถ้า job เป็นสีเขียวทั้งหมด แปลว่า deploy สำเร็จ